

Multi-Modal Deep Learning

Lecture Series | Summer 2025

Institute for AI and Informatics in Medicine

24.06.2025

MOTIVATION

- State-of-the-art multimodal transformers are very, very large

22 Jul 2020

Language Models are Few-Shot Learners

Tom B. Brown*	Benjamin Mann*	Nick Ryder*	Melanie Subbiah*	
Jared Kaplan ¹	Prafulla Dhariwal	Arvind Neelakantan	Pranav Shyam	Girish Sastry
Amanda Askell	Sandhini Agarwal	Ariel Herbert-Voss	Gretchen Krueger	Tom Henighan
Rewon Child	Aditya Ramesh	Daniel M. Ziegler	Jeffrey Wu	Clemens Winter
Christopher Hesse	Mark Chen	Eric Sigler	Mateusz Litwin	Scott Gray
Benjamin Chess	Jack Clark	Christopher Berner		

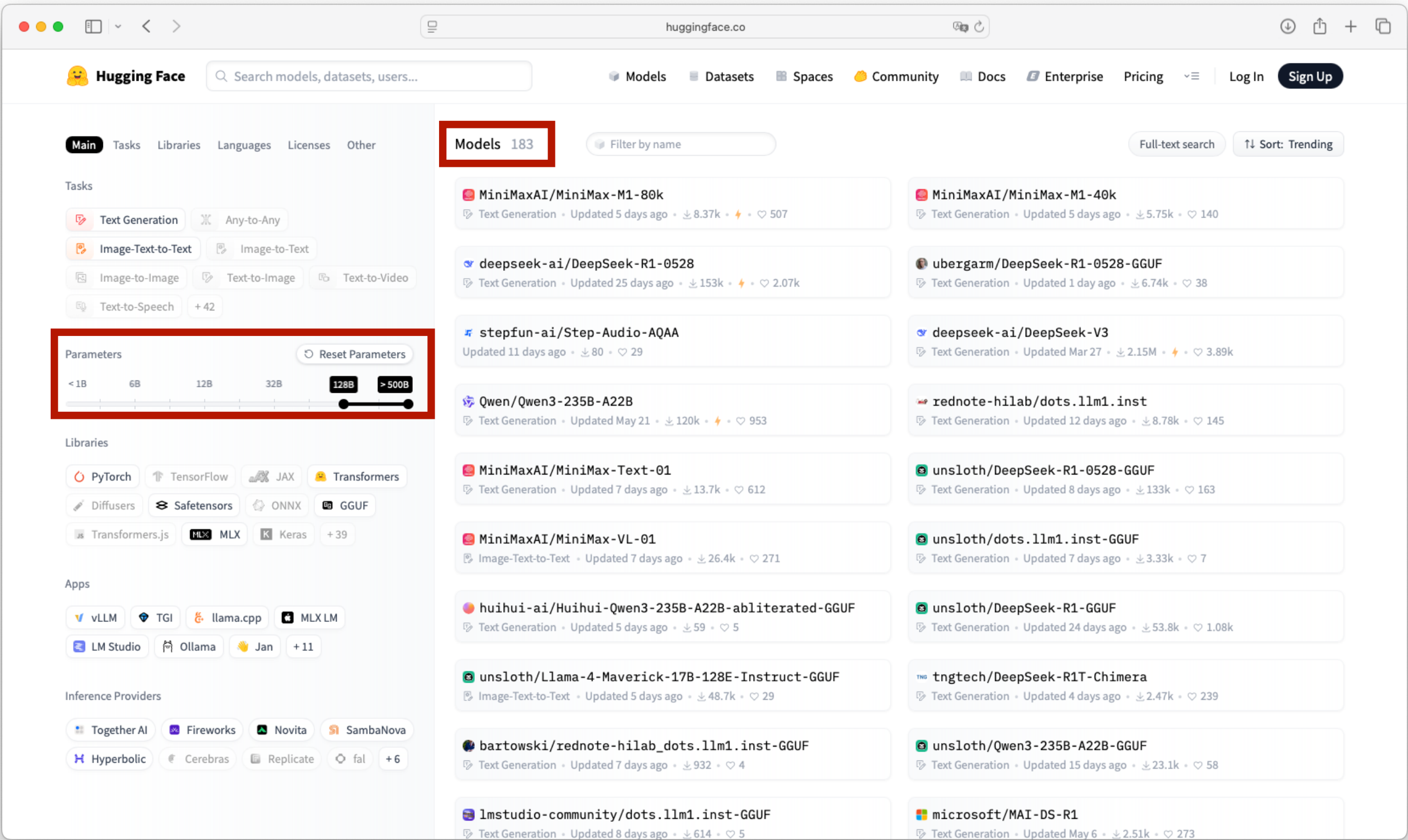
a few examples or from simple instructions – something which current NLP systems still largely struggle to do. Here we show that **scaling up language models greatly improves task-agnostic, few-shot performance**, sometimes even reaching competitiveness with prior state-of-the-art fine-tuning approaches. **Specifically, we train GPT-3, an autoregressive language model with 175 billion parameters, 10x more than any previous non-sparse language model**, and test its performance in the few-shot setting. For all tasks, GPT-3 is applied without any gradient updates or fine-tuning, with tasks and few-shot demonstrations specified purely via text interaction with the model. GPT-3

few-shot performance, sometimes even reaching competitiveness with prior state-of-the-art fine-tuning approaches. Specifically, we train GPT-3, an autoregressive language model with 175 billion parameters, 10x more than any previous non-sparse language model, and test its performance in the few-shot setting. For all tasks, GPT-3 is applied without any gradient updates or fine-tuning, with tasks and few-shot demonstrations specified purely via text interaction with the model. GPT-3 achieves strong performance on many NLP datasets, including translation, question-answering, and cloze tasks, as well as several tasks that require on-the-fly reasoning or domain adaptation, such as unscrambling words, using a novel word in a sentence, or performing 3-digit arithmetic. At the same time, we also identify some datasets where GPT-3's few-shot learning still struggles, as well as some datasets where GPT-3 faces methodological issues related to training on large web corpora. Finally, we find that GPT-3 can generate samples of news articles which human evaluators have difficulty distinguishing from articles written by humans. We discuss broader societal impacts of this finding and of GPT-3 in general.

^{*}Equal contribution

¹Johns Hopkins University, OpenAI

Author contributions listed at end of paper.



MOTIVATION

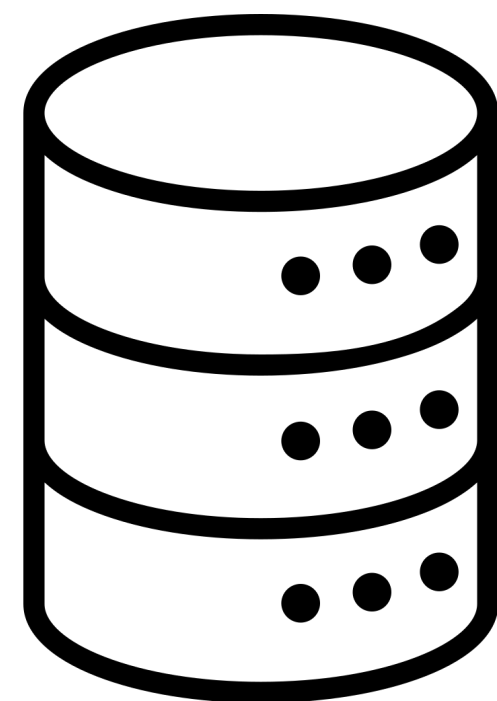
- State-of-the-art multimodal transformers are very, very large
- Training requires parallel use of multiple GPUs with large VRAM
- Even storing model checkpoints is difficult

GPT-3	
Number of parameters	175,000,000,000
Required VRAM	1.2 TB
Estimated training time	355 GPU years
Size per check point	350 GB

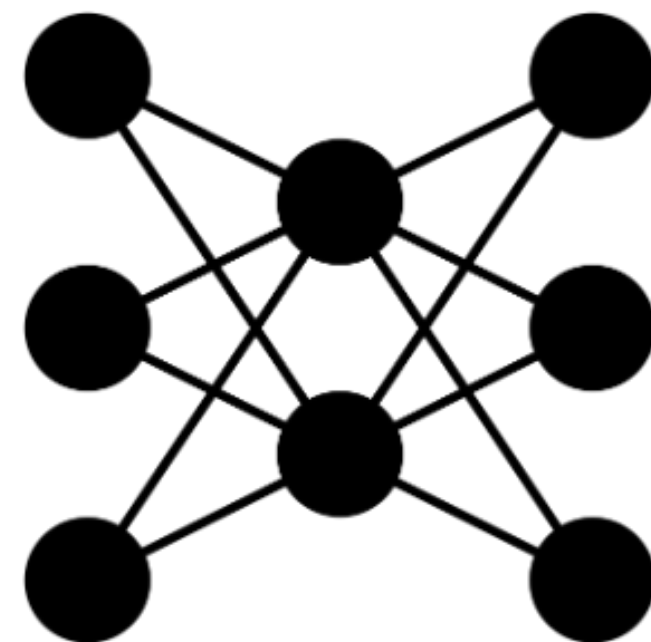
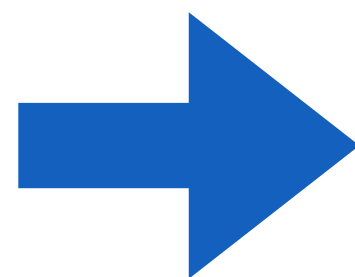
REAL-WORLD IMPLICATIONS

- Large storage and training requirements make common tasks very difficult:
 - Updating a model with more data

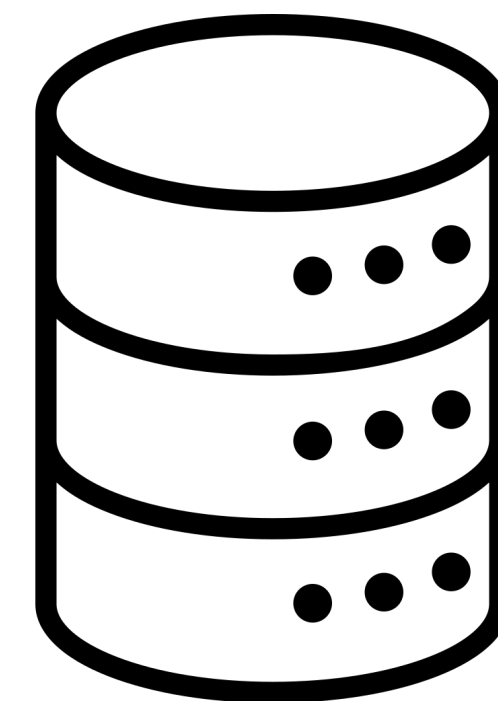
Training



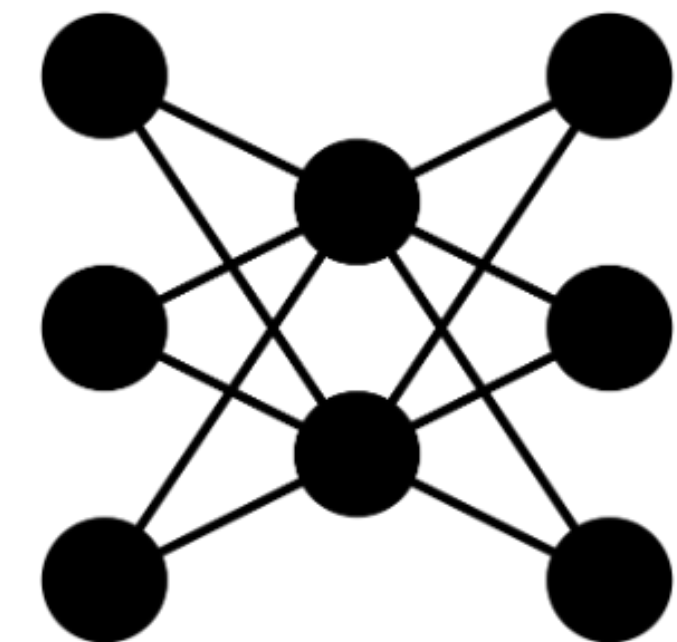
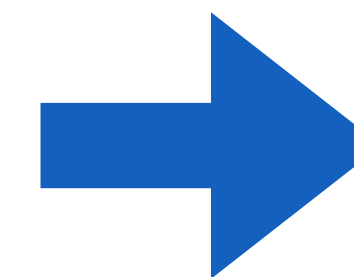
Encyclopedia Britannica



Fine-tuning



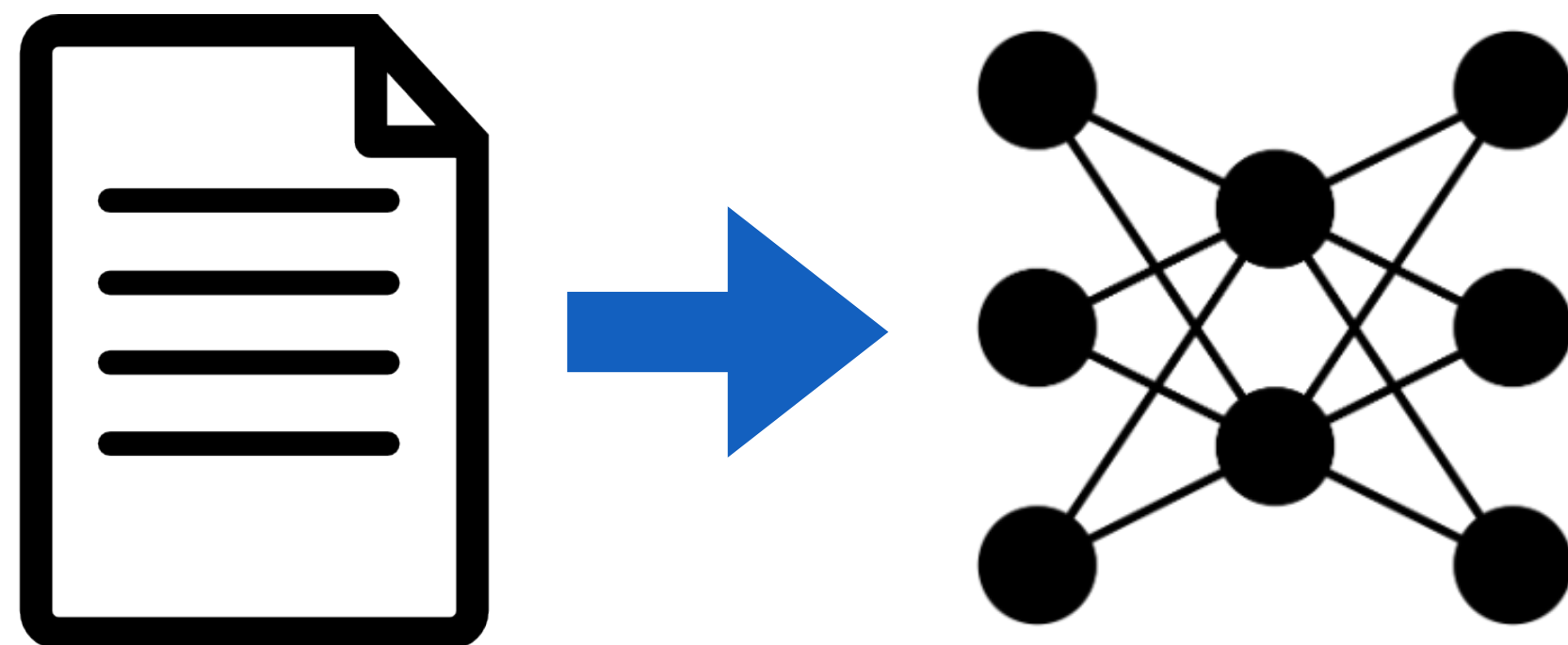
Wikipedia



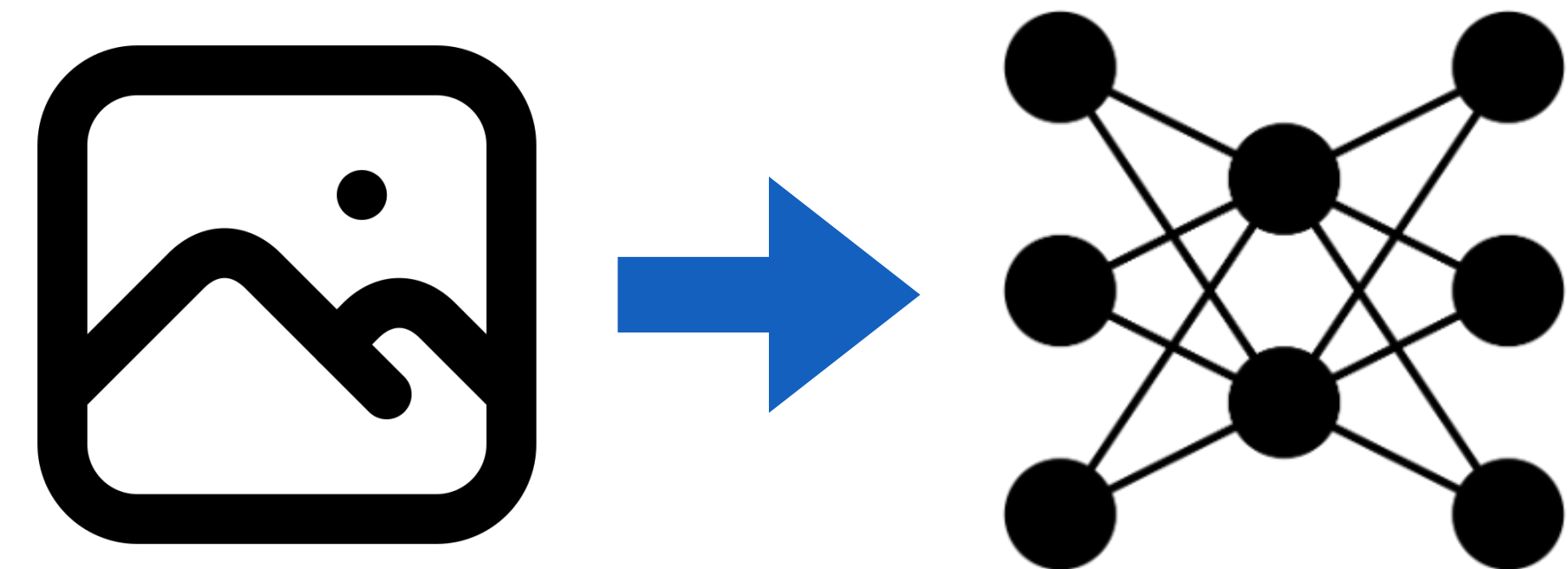
REAL-WORLD IMPLICATIONS

- Large storage and training requirements make common tasks very difficult:
 - Updating a model with more data
 - Linking a model to a new modality

Training



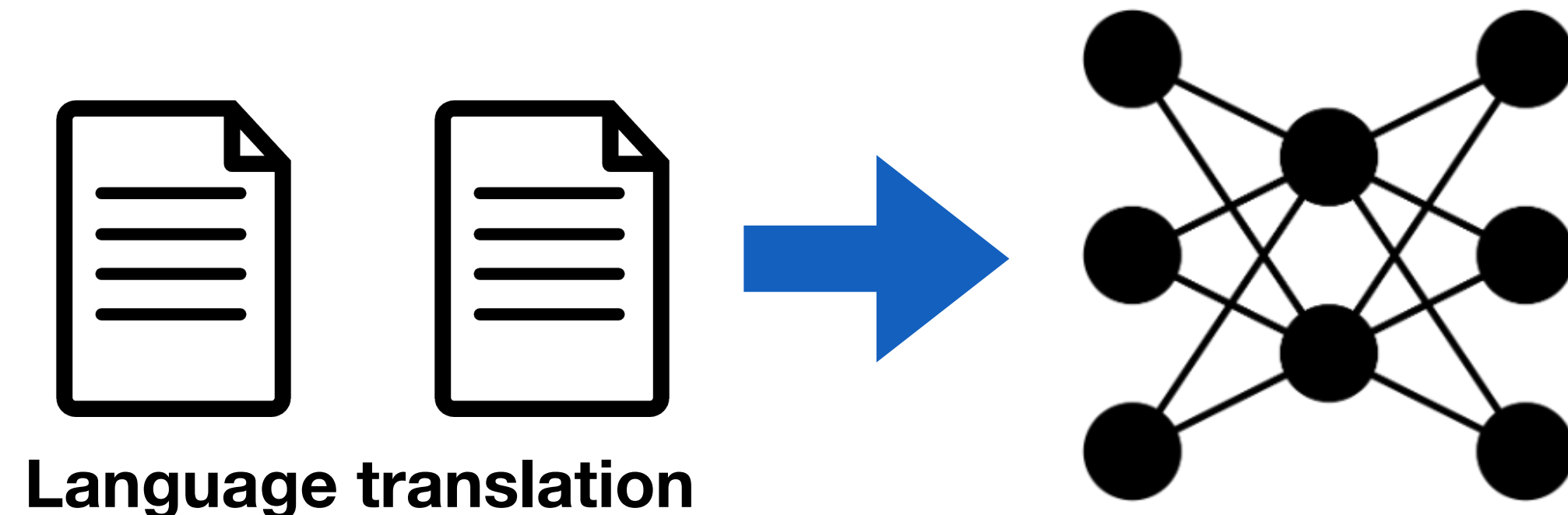
Fine-tuning



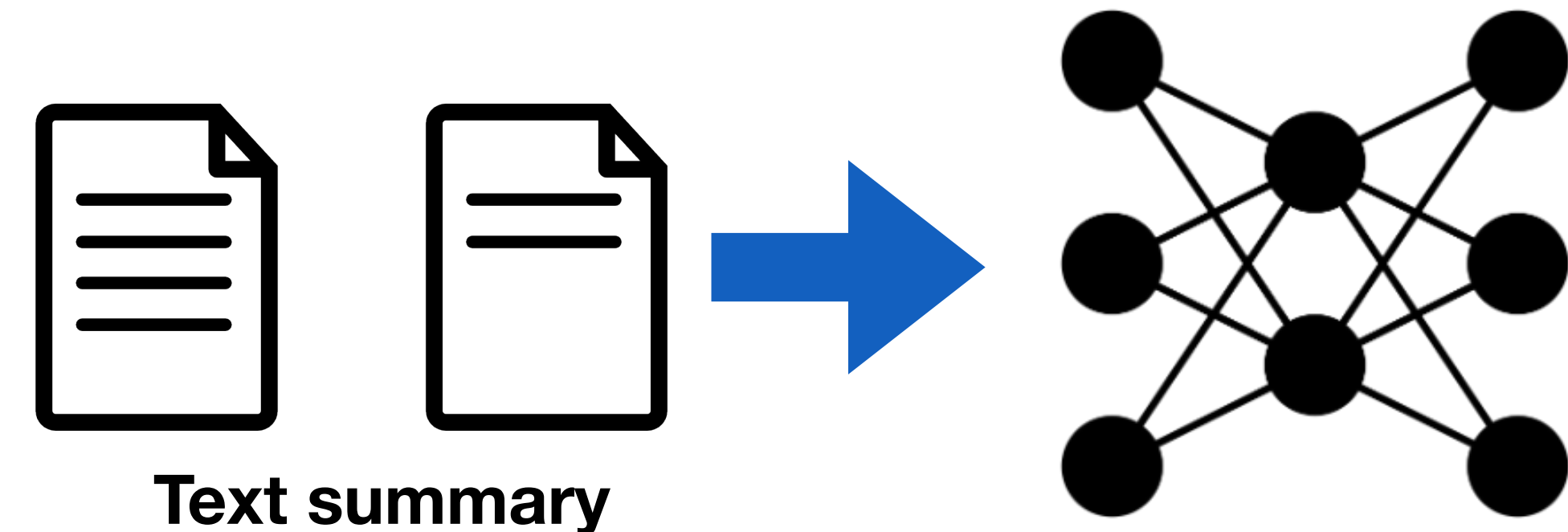
REAL-WORLD IMPLICATIONS

- Large storage and training requirements make common tasks very difficult:
 - Updating a model with more data
 - Linking a model to a new modality
 - Tuning model for different tasks

Model flavor 1

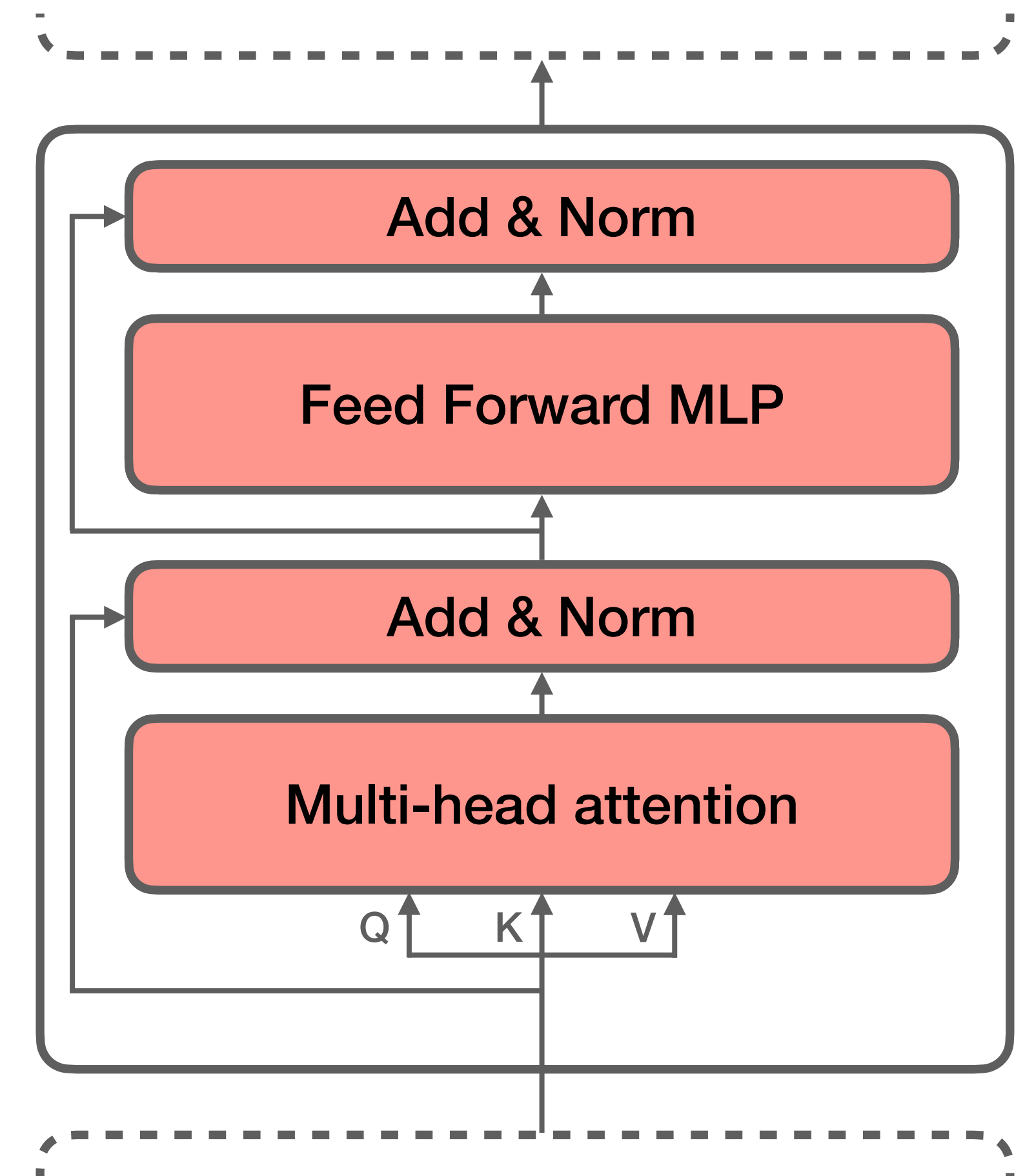


Model flavor 2



FULL FINE-TUNING

- Re-optimize all networks weights
- + Conceptionally simple
- + Should intuitively results in highest performance
- Computationally infeasible in many situations
- Catastrophic forgetting may occur, especially with small fine-tuning datasets



PARAMETER-EFFICIENT FINE-TUNING

Definition

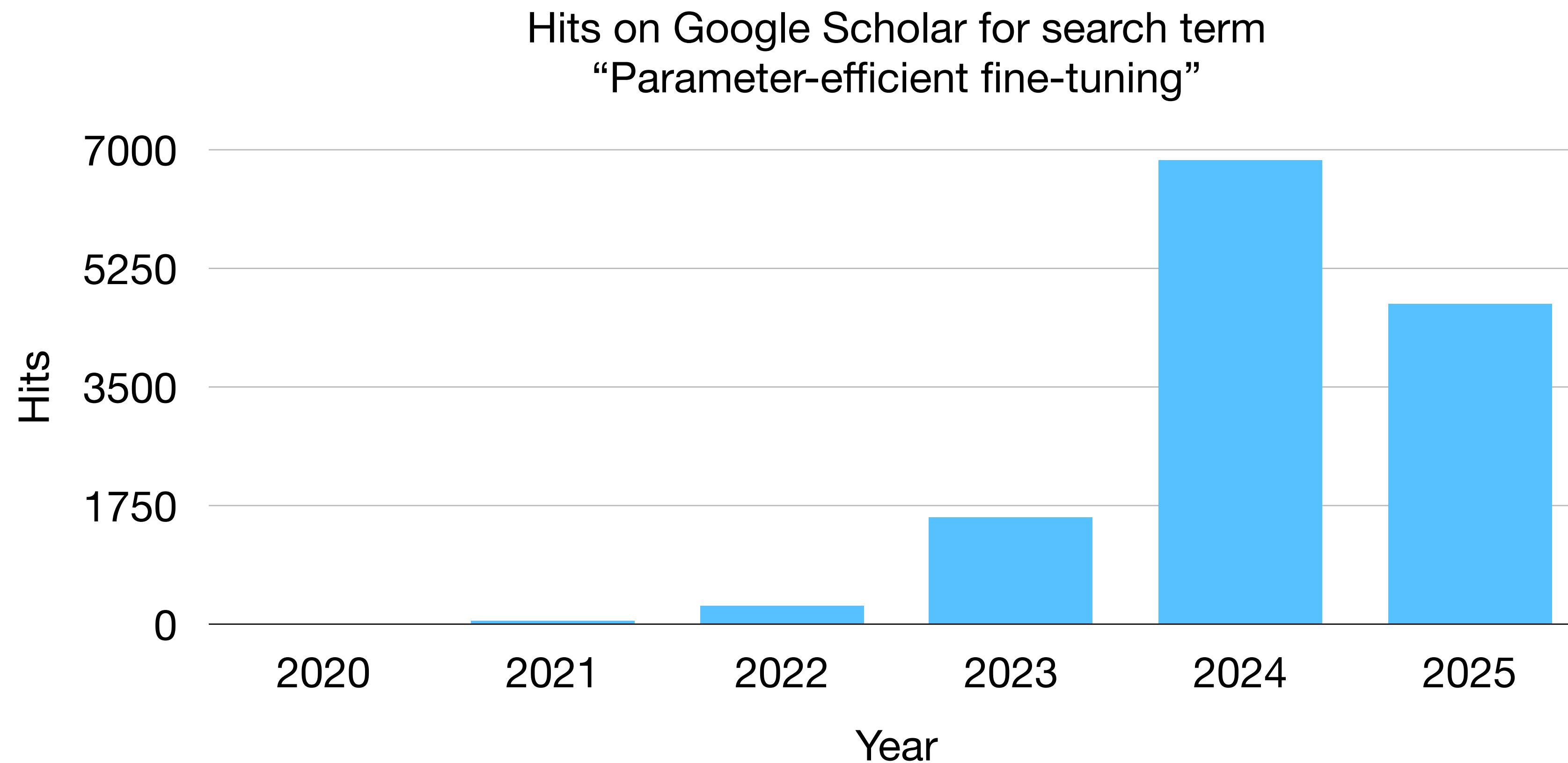
*"Parameter-efficient fine-tuning (PEFT) aims to **adapt large pre-trained neural networks to specific tasks with significantly fewer trainable parameters** compared to full fine-tuning in order to reduce computational cost, allowing for faster adaptation and deployment."*

CONTENT OF THIS LECTURE

- Will introduce different types of parameter-efficient fine-tuning:
 - Selective parameter fine-tuning
 - Adapters
 - Prompt tuning
 - Reparameterization-based methods
- Focus on the main intuition of each type

DISCLAIMER

- This is a very fast moving field of research

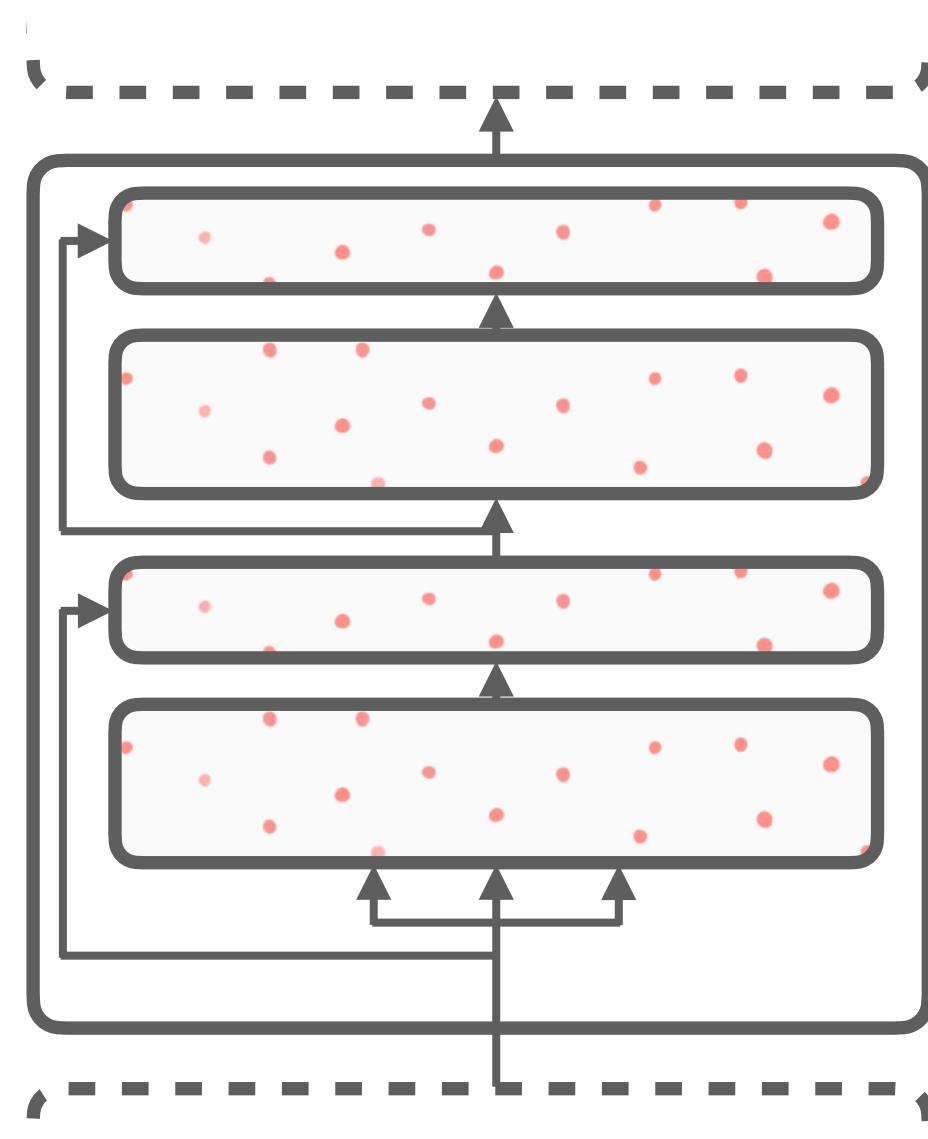


DISCLAIMER

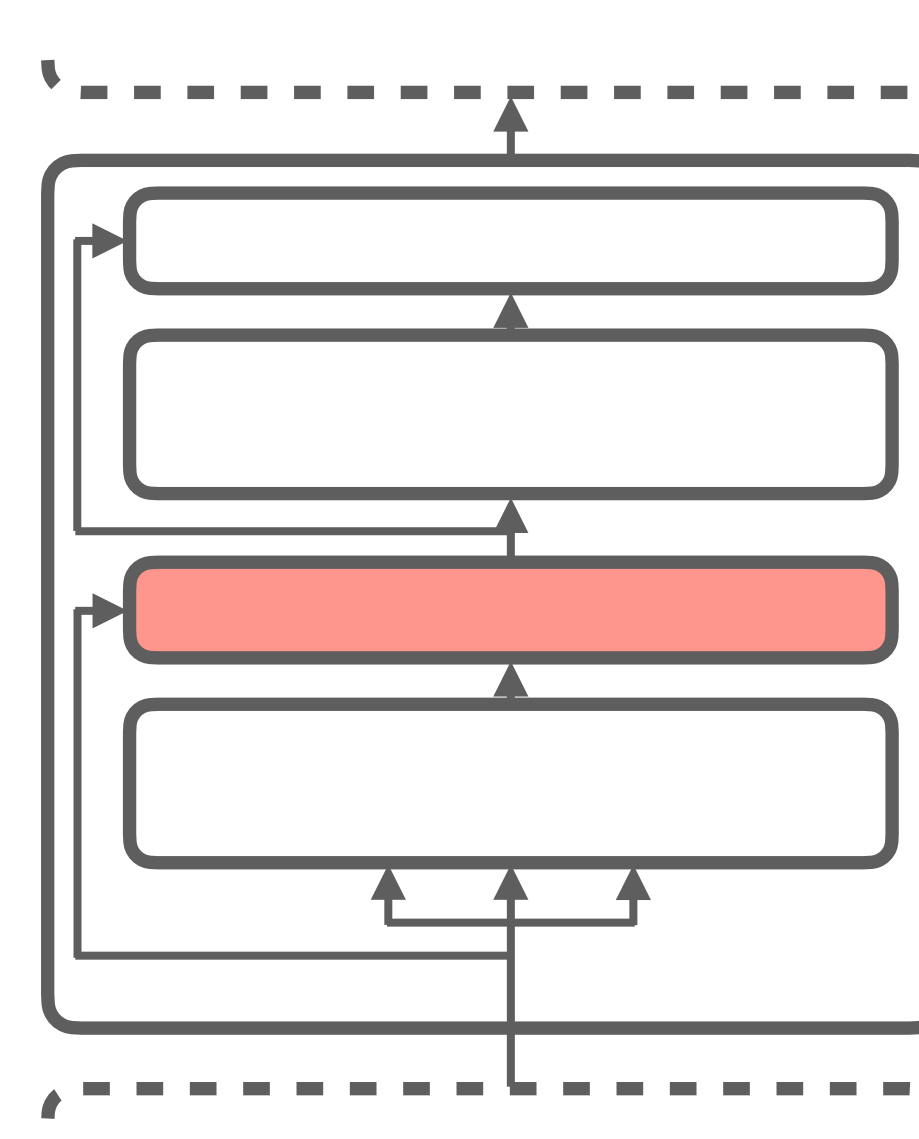
- This is a very fast moving field of research
- Effectiveness of methods may vary dependent on:
 - Model size
 - Available compute
 - Training and fine-tuning data
 - Downstream task
- Highest performing parameter-efficient fine-tuning strategies are increasingly becoming corporate trade secrets

SELECTIVE PARAMETER FINE-TUNING

- Selective methods only fine-tune a subset of the model's parameters
- Typically between 0.01% and 1% of parameters tuned
- Methods mainly differ how the tunable parameters are selected



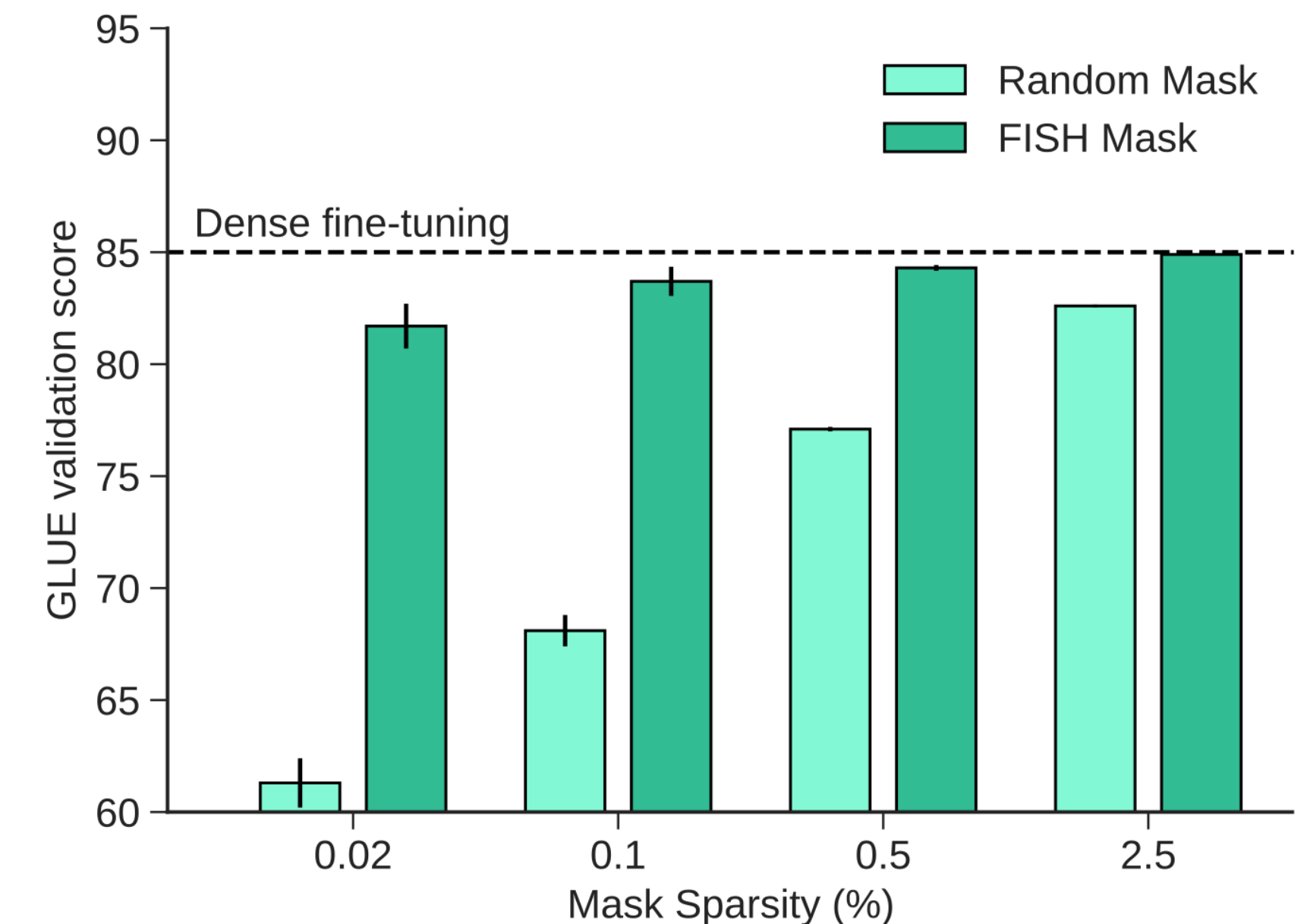
Unstructured selection



Structured selection

FISHMASK

- Identification of most important parameters via Fisher information
- Fisher information measures impact of weight change on network output
- + Principled selection of updatable parameters
- + Allows use of very sparse masks
- Fisher information can only be approximated
- Calculation of Fisher information comes with an computational overhead
- Requires hardware support for sparse operations

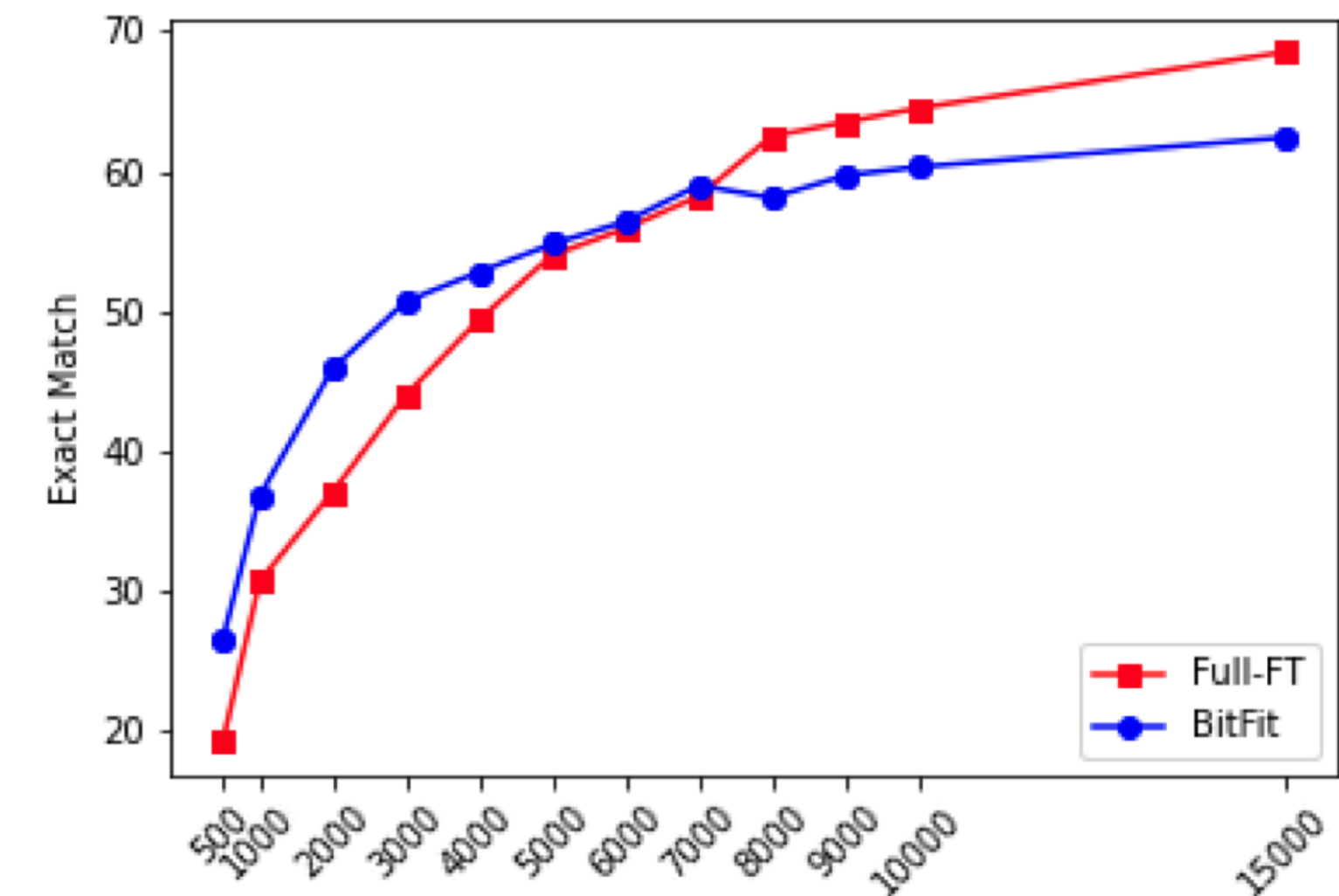


DIFFPRUNING

- Use of back propagation to find most relevant network parameters
- Learnable binary mask in form of a differentiable L_0 Norm
- + Comparable performance while updating only 0.5% of parameters
- Requires substantial amount of memory to learn mask
- Inefficient on modern hardware

BITFIT

- BitFit only fine-tunes the bias terms of a transformer
- Query and “middle-of-MLP” Bias terms are shown to be most impactful
- + Only 0.05% of all parameters are updated
- + Surprisingly good performance on medium-size networks
- Diminished benefit in large networks
- Some modern transformer architectures (e.g. LLaMA) do not have bias terms

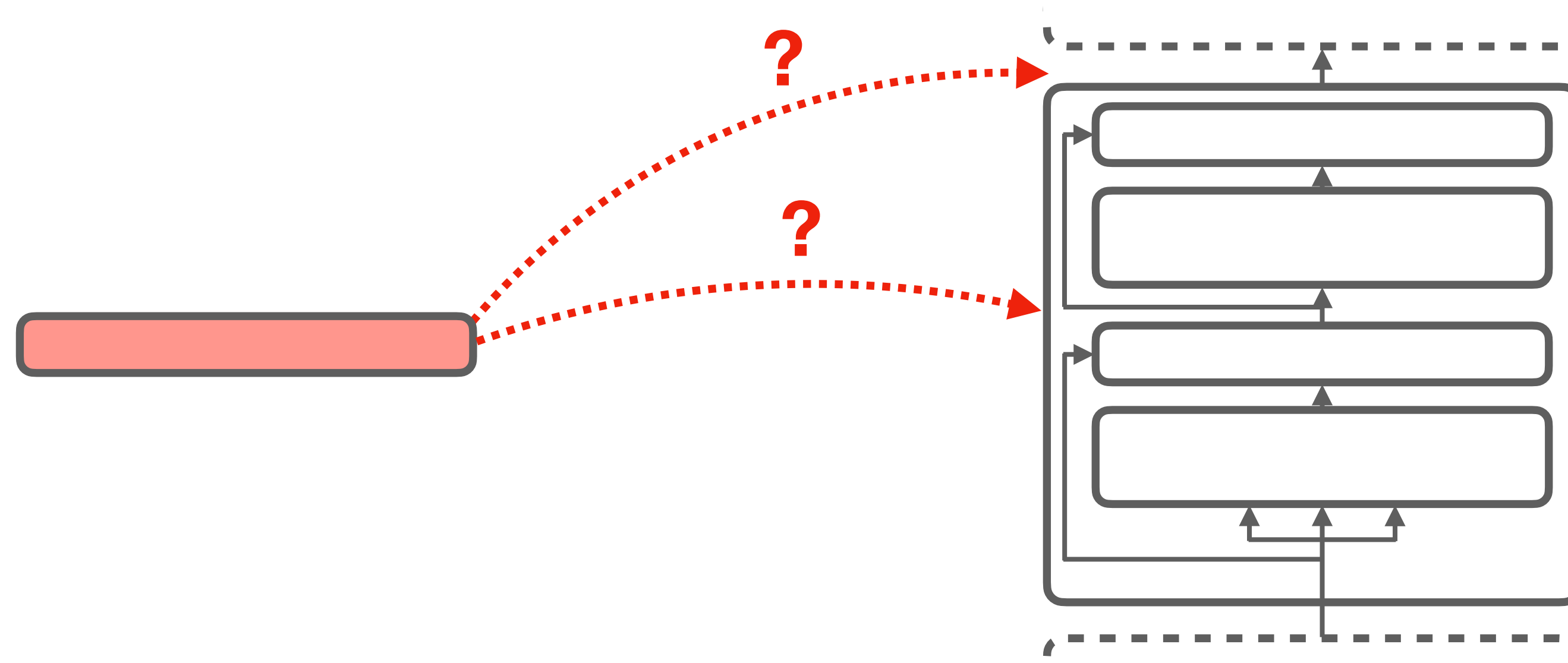


SELECTIVE PARAMETER FINE-TUNING

- Selective methods only fine-tune a subset of the model's parameters
- Typically between 0.01% and 1% of parameters tuned
- Methods mainly differ how the tunable parameters are selected
 - Based on weight values or information content (e.g. FishMask)
 - Learn most influential parameters via backpropagation (e.g. DiffPruning)
 - According to role in network (e.g. BitFit)

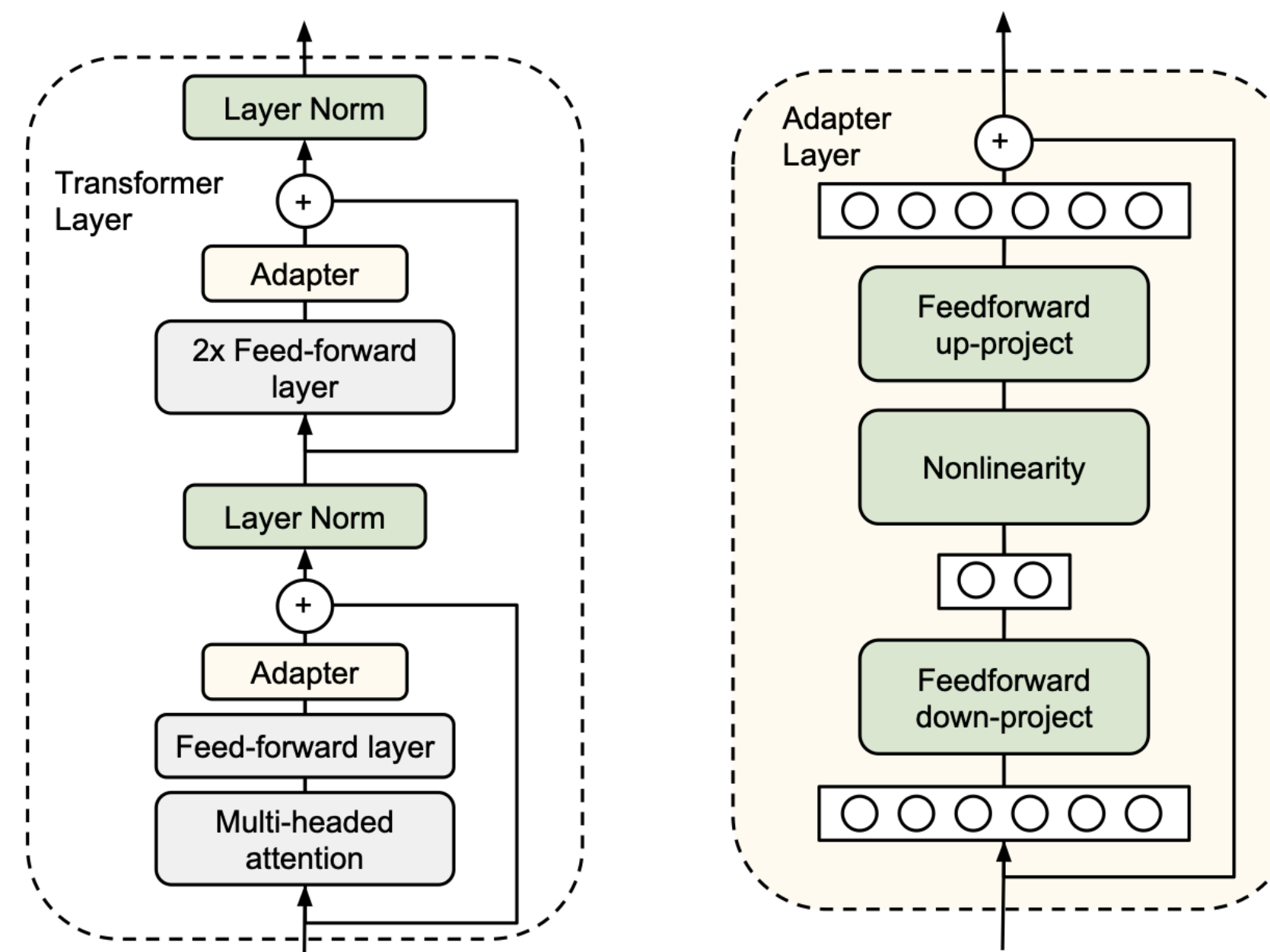
ADAPTERS

- Strategically placed trainable parameters
- Number, position, and capacity of added layers has to be decided



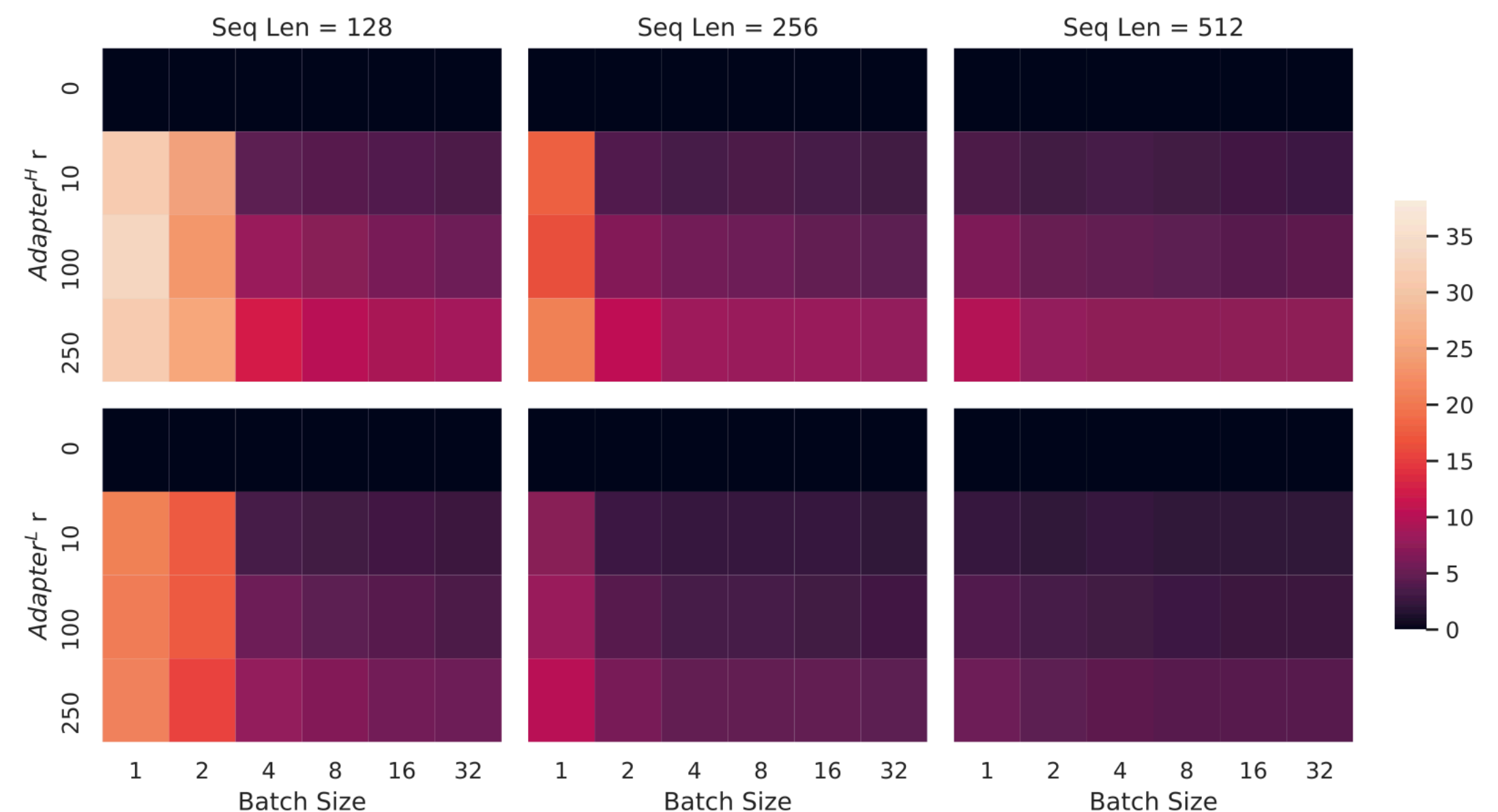
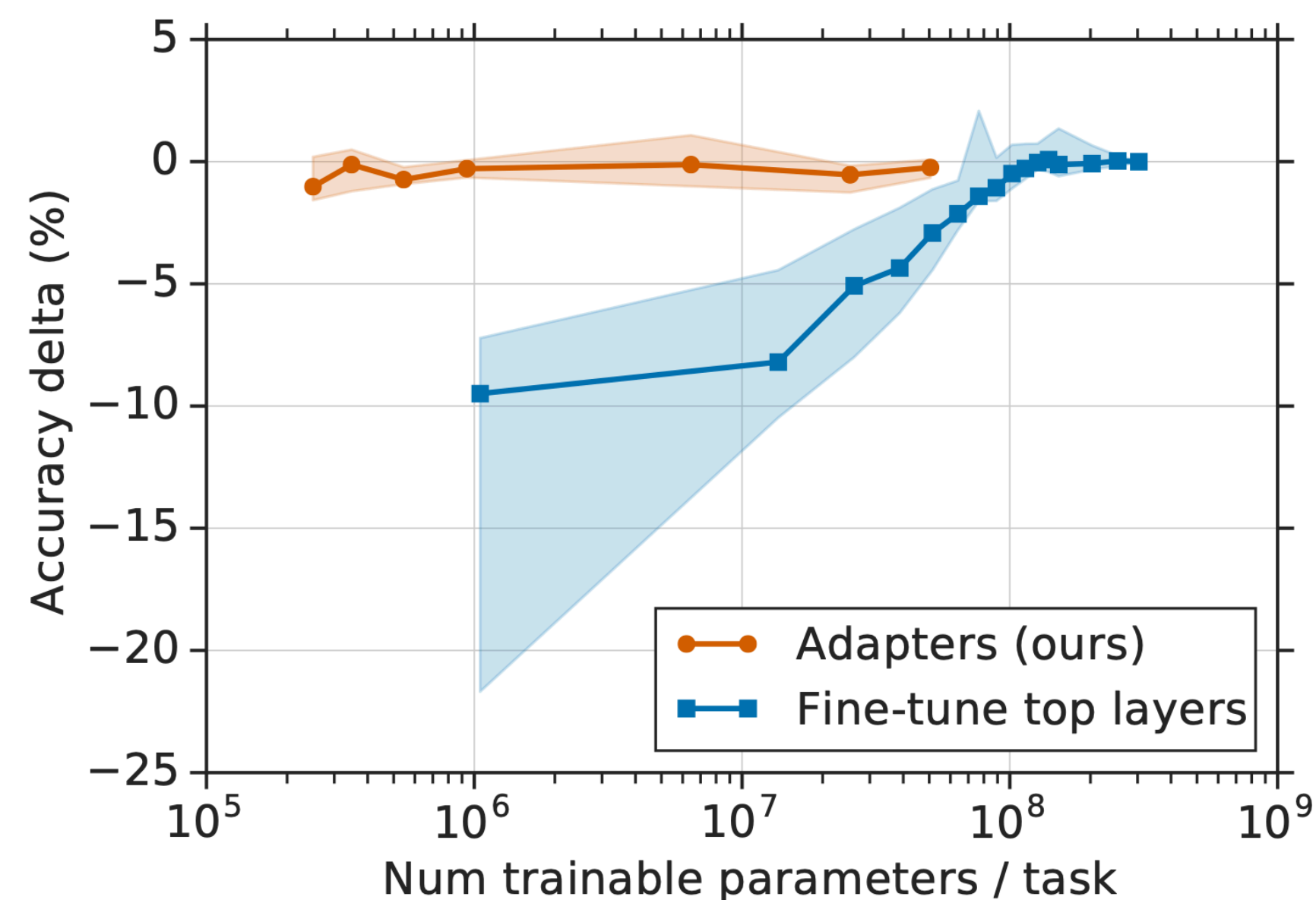
BASIC SERIAL ADAPTER

- Adapter layers added after attention block and feed-forward MLP (Later variants only include a single block after the self-attention layer)
- Encoder-decoder structure of adapter to reduce complexity
- Skip connections allow circumventing adapter layers



BASIC SERIAL ADAPTER

- + Good performance across network scales
- Increased inference overhead due to sequential nature of network
- Unable to fully exploit hardware parallelism due to repeated broadcasting



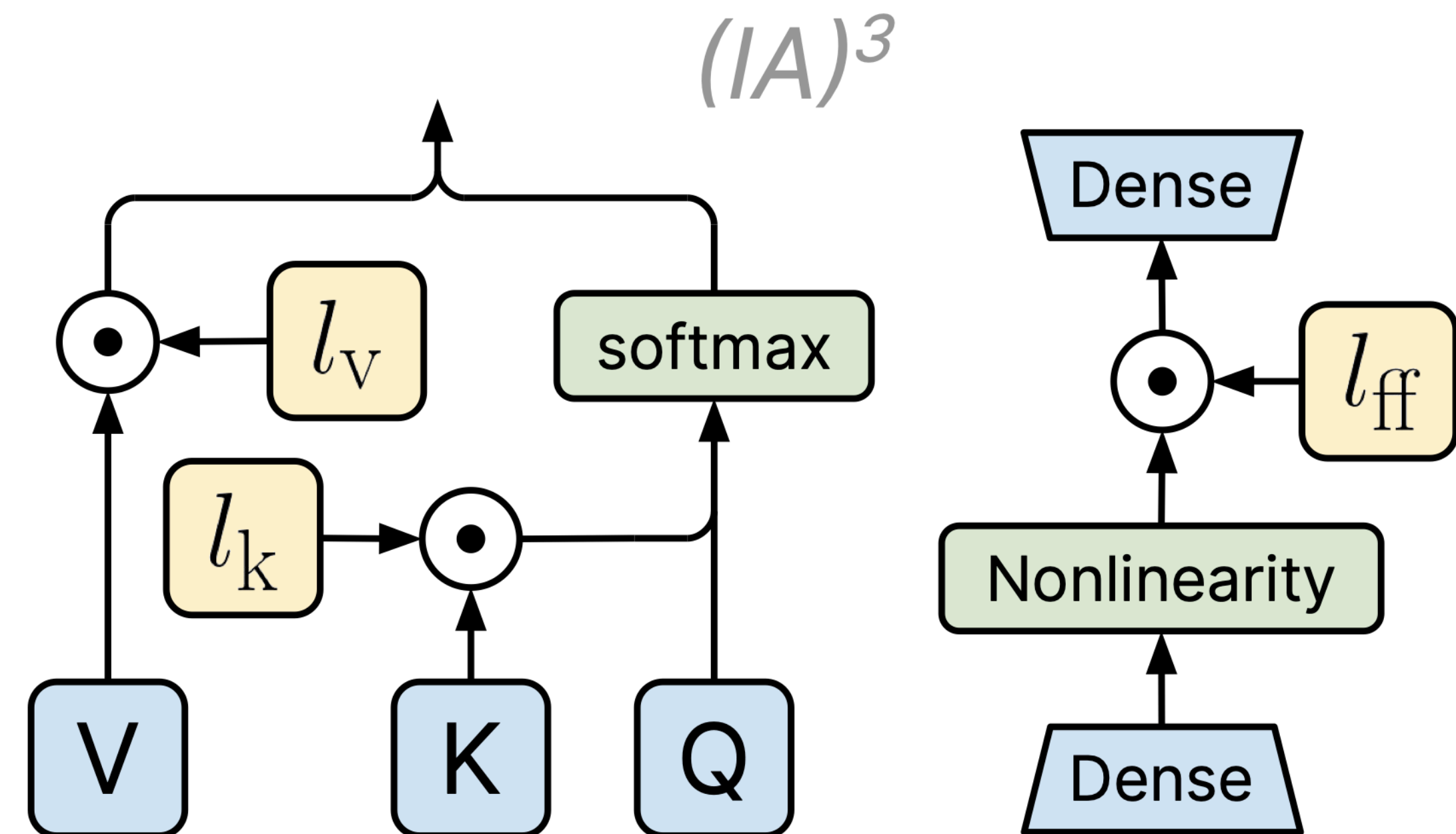
$(IA)^3$

- Learnable scaling vectors of vector and key attention matrices and FFN
- Hybrid between selective and additive PEFT

+ Very few trainable parameters

+ Influences large parts of network

- Limited expressibility



PROMPTS IN TRANSFORMERS

- Prefix at the beginning of the input sequence
- Prominent examples are chain-of-thought prompting or in-context learning

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A: The answer (arabic numerals) is

(Output) 8 **X**

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

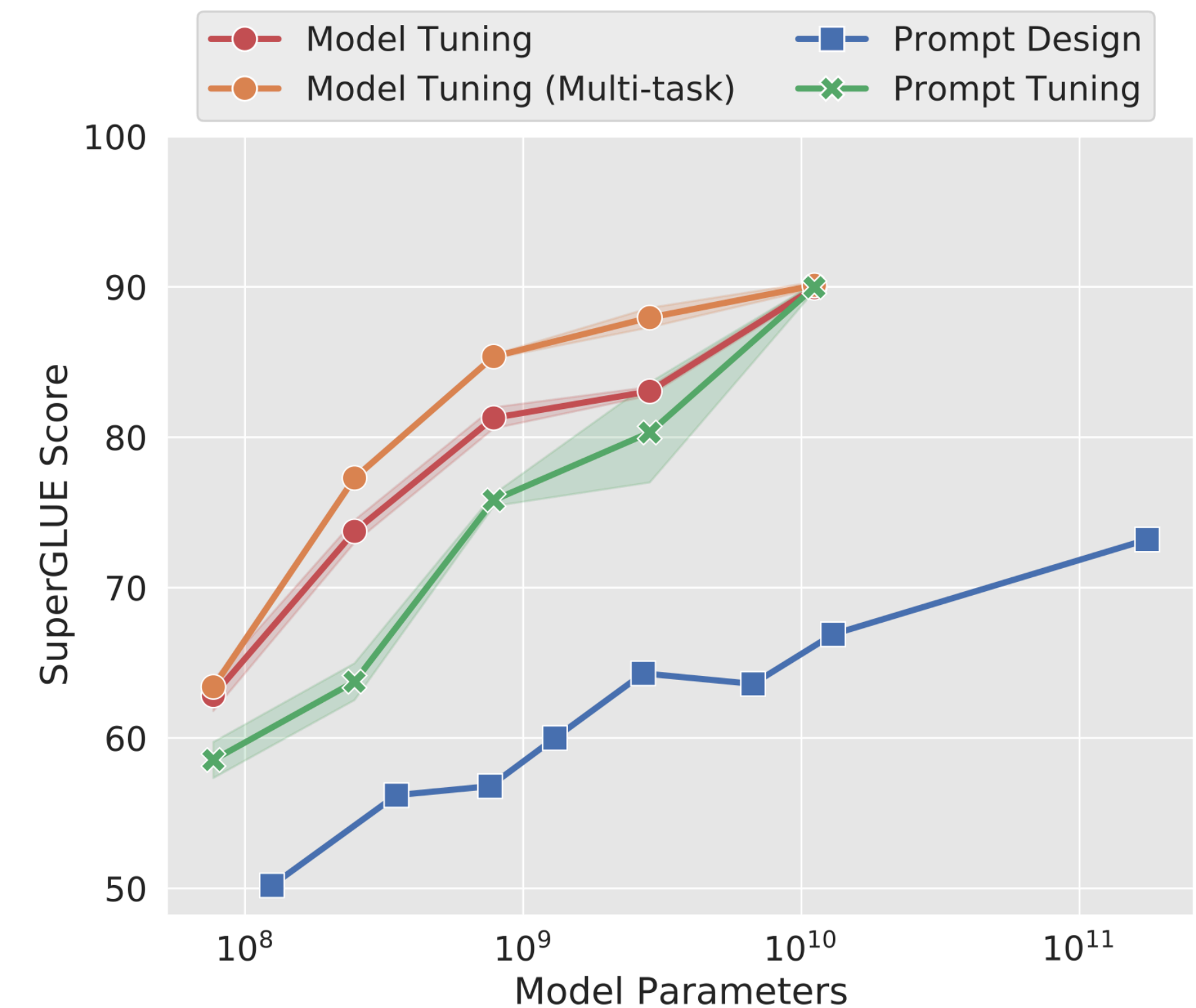
A: **Let's think step by step.**

(Output) *There are 16 balls in total. Half of the balls are golf balls. That means that there are 8 golf balls. Half of the golf balls are blue. That means that there are 4 blue golf balls. ✓*

- Prompt tuning aims to learn this prefix instead of using crafted prompts

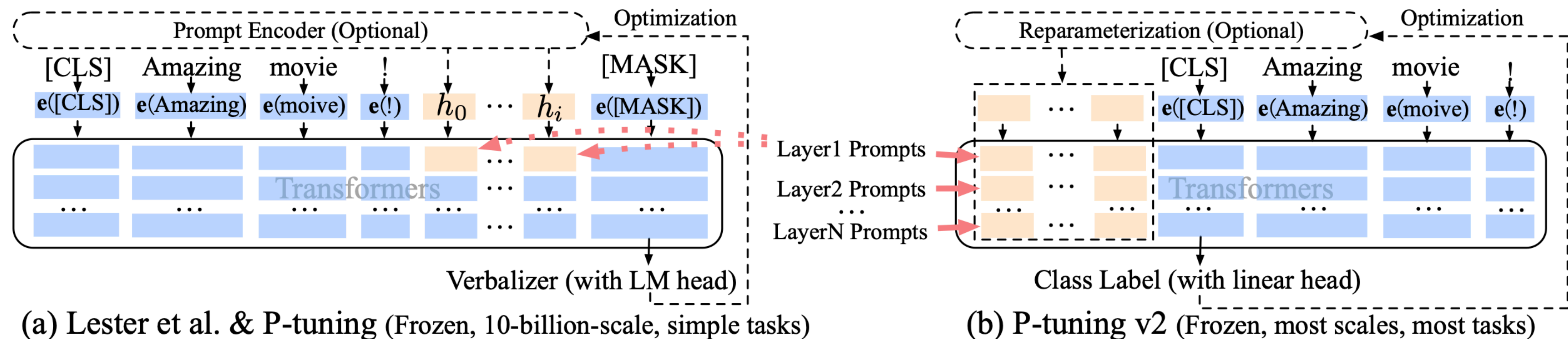
PROMPT TUNING

- Prepend learnable soft embeddings at the beginning of input sequence
- Direct optimization through gradient descent
- + Very parameter- and storage-efficient
- + Excellent performance also in large models
- Difficult to tune
- Slow to converge
- Reduces available sequence length



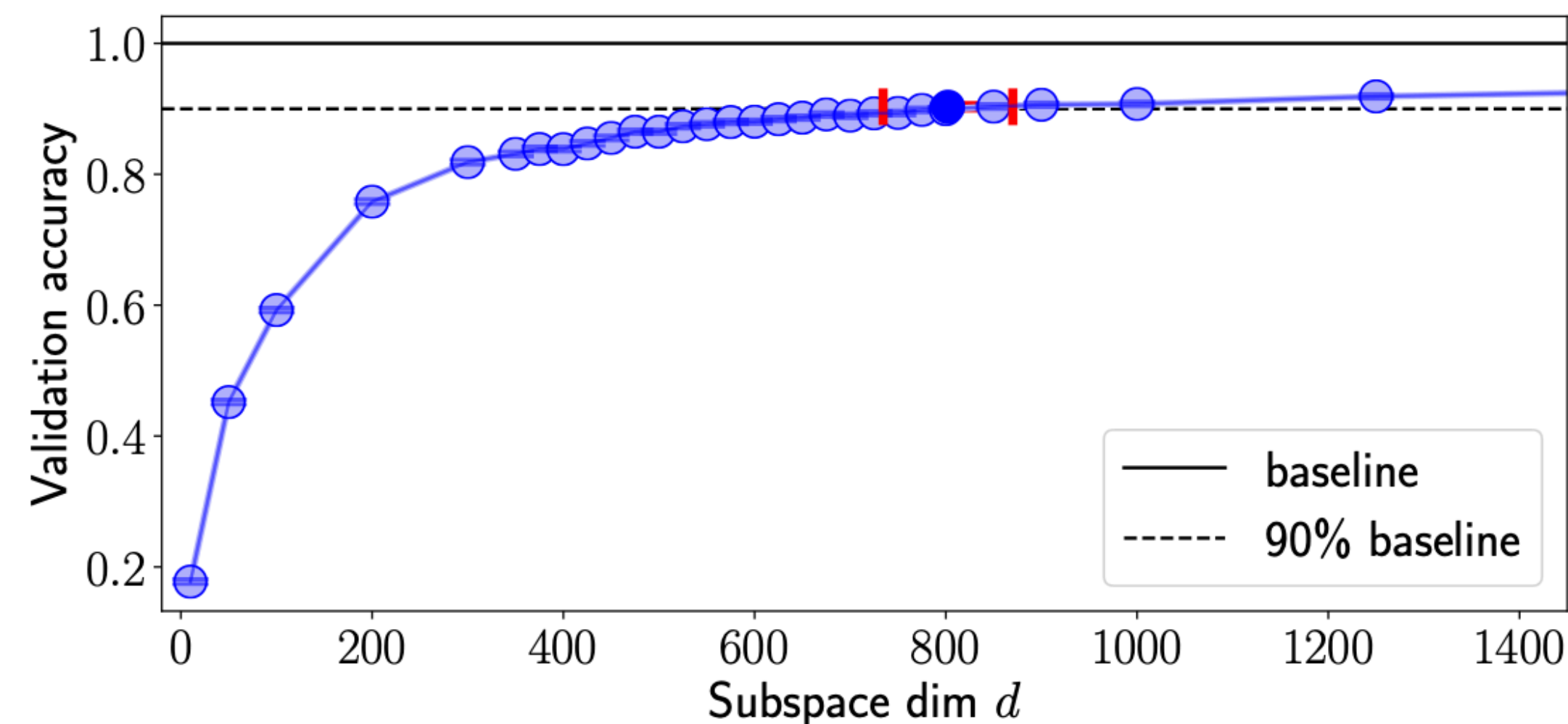
P-TUNING V2

- Addresses slow convergence of prompt tuning with two main changes
- Same prefix of soft tokens is added to the hidden states of all layers
- Instead of learning soft prompt tokens directly, their generation is parameterized by a small neural network



TRANSFORMER HAVE LOW INTRINSIC DIMENSION

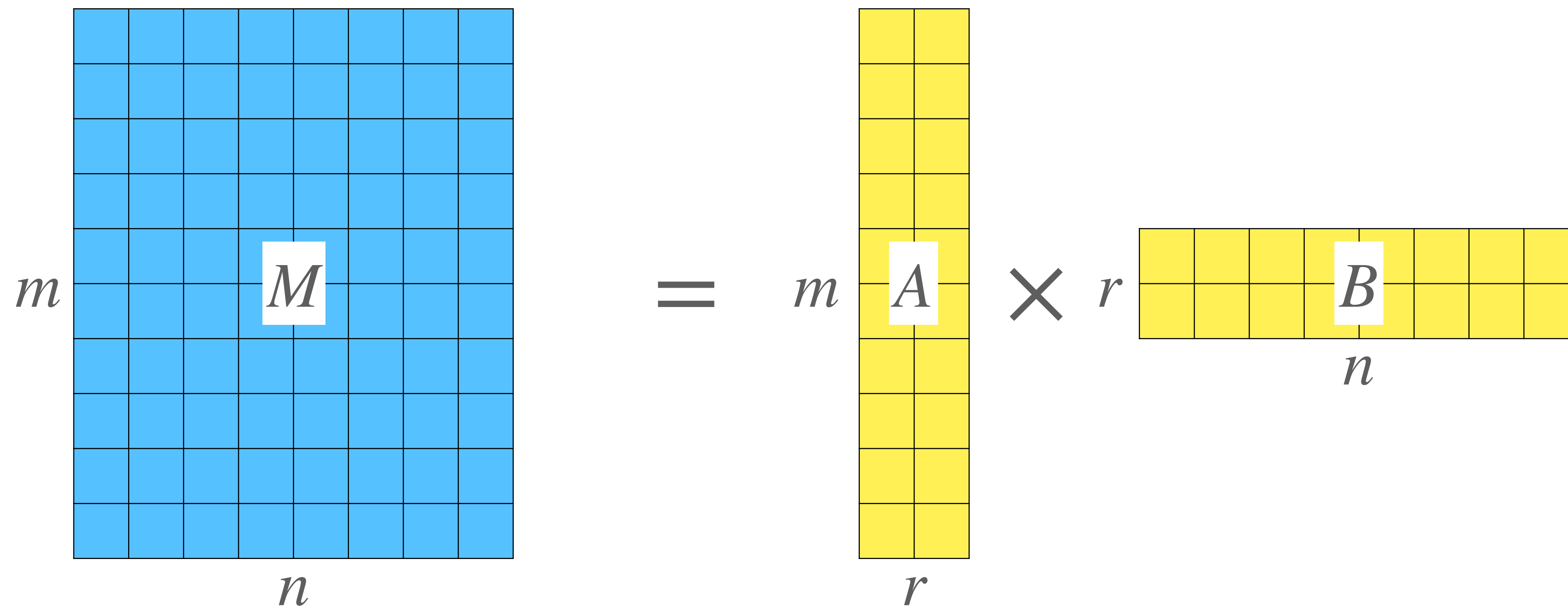
- Intrinsic dimensionality can be understood as the minimum number of parameters d needed to replicate a certain level of performance achieved with a larger number of parameters D
- Empiric experiments show that neural networks - including large LLMs - have low intrinsic dimensionality



Approximation of a fully-connected neural network with 199,210 parameters

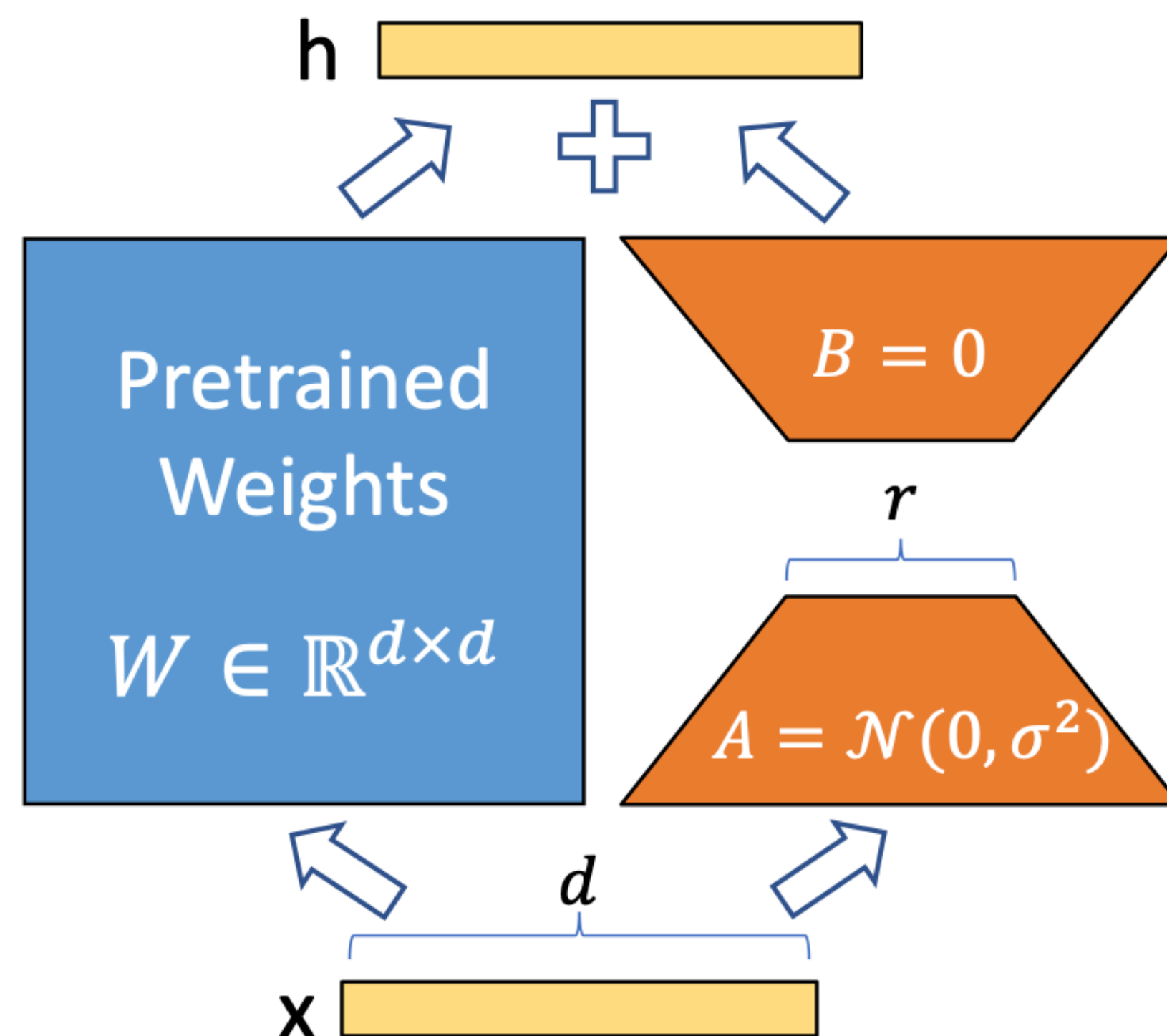
LOW-RANK MATRIX DECOMPOSITION

- The rank of a matrix is the maximum number of linearly independent rows or columns
- A matrix $M \in \mathbb{R}^{m \times n}$ with rank r can be approximated by a multiplication of two matrices $A \in \mathbb{R}^{m \times r}$, $B \in \mathbb{R}^{r \times n}$



LOW-RANK ADAPTATION

- Low-rank adaptation (LoRA) reparameterizes the weight updates to a matrix W , ΔW , as a learnable low-rank approximation
- Only $2 \cdot d \cdot r$ parameters have to be stored instead of $d \cdot d$ parameters



$$h = Wx + \Delta Wx$$

$$\Delta W = AB,$$
$$A \in \mathbb{R}^{d \times r}, B \in \mathbb{R}^{r \times d}$$

LOW-RANK ADAPTATION

- Key and Value matrices are typically chosen for updates
- Weight updates are scaled with simple scalar (comparable with learning rate hyperparameter)
- For d of 12,288, r can be as low as 4, 2, or even 1

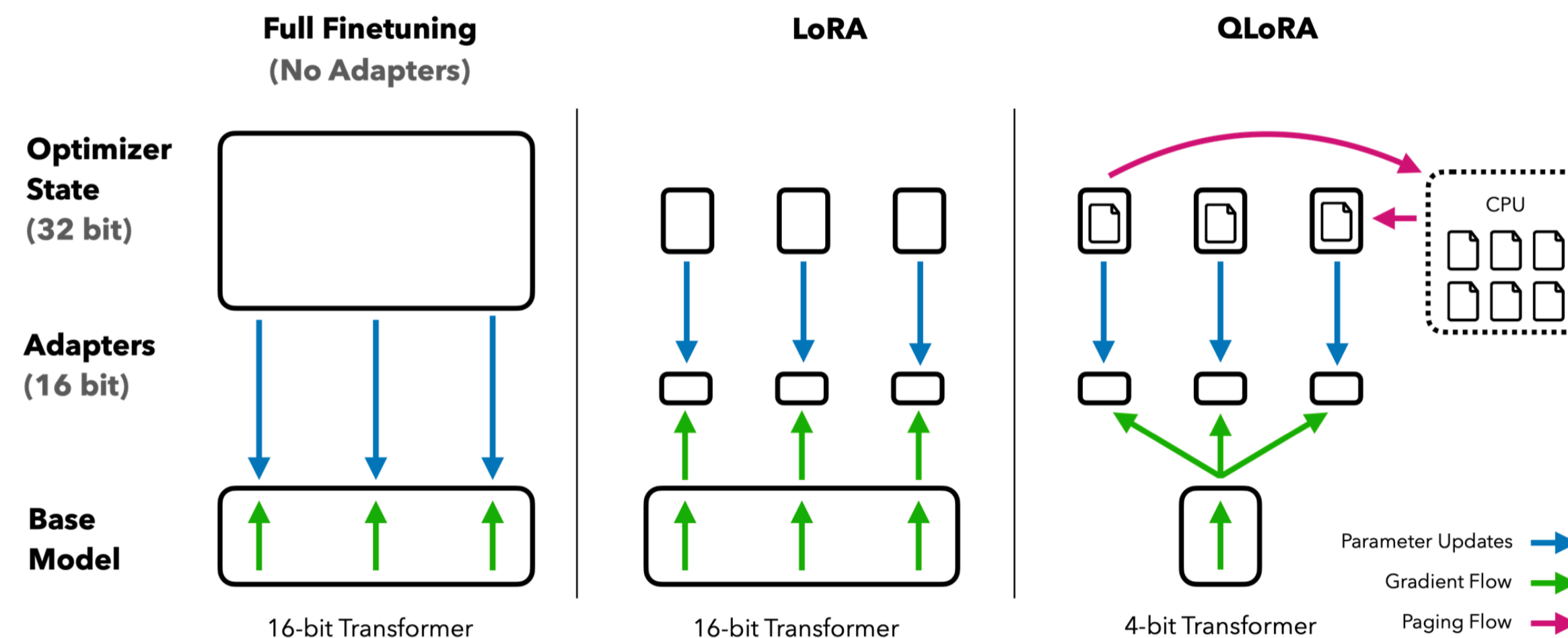
	# of Trainable Parameters = 18M						
Weight Type Rank r	W_q 8	W_k 8	W_v 8	W_o 8	W_q, W_k 4	W_q, W_v 4	W_q, W_k, W_v, W_o 2
WikiSQL ($\pm 0.5\%$)	70.4	70.0	73.0	73.2	71.4	73.7	73.7
MultiNLI ($\pm 0.1\%$)	91.0	90.8	91.0	91.3	91.3	91.3	91.7

LOW-RANK ADAPTATION

- + Allows updating of large model weight matrices (attention modules)
- + Easy to tune and good convergence
- + Natural generalization of full model fine-tuning
- + Weight updates can be quickly swapped and updated
- Does not work well for small models

QLoRA

- 4-bit quantization of backbone model
 - LoRA gradient updates requires temporary conversion to full precision
- + Substantial reduction of main model memory footprint
- Model quantization may reduce performance



SUMMARY

- Introduced the following parameter-efficient fine-tuning strategies
 - Selective parameter fine-tuning
 - Adapters
 - Prompt tuning
 - Reparameterization-based methods
- There are dozens of minor variations to each method
- Hybrid methods have been introduced as well

DISCLAIMER

- This is a very fast moving field of research
- Effectiveness of methods may vary dependent on:
 - Model size
 - Available compute
 - Training and fine-tuning data
 - Downstream task
- Highest performing parameter-efficient fine-tuning strategies are increasingly becoming corporate trade secrets

HUGGINGFACE PEFT LIBRARY

• PEFT

Search documentation

V0.15.0 EN 18,836

ADAPTERS

AdaLoRA

IA3

Llama-Adapter

LoHa

LoKr

LoRA

X-LoRA

LyCORIS

Multitask Prompt Tuning

OFT

BOFT

Polytropon

P-tuning

Prefix tuning

Prompt tuning

Layernorm tuning

VeRA

FourierFT

VB-LoRA

HRA

CPT

Bone

Trainable Tokens

LoRA

Low-Rank Adaptation (LoRA) is a PEFT method that decomposes a large matrix into two smaller low-rank matrices in the attention layers. This drastically reduces the number of parameters that need to be fine-tuned.

The abstract from the paper is:

We propose a neural language modeling system based on low-rank adaptation (LoRA) for speech recognition output rescoring. Although pretrained language models (LMs) like BERT have shown superior performance in second-pass rescoring, the high computational cost of scaling up the pretraining stage and adapting the pretrained models to specific domains limit their practical use in rescoring. Here we present a method based on low-rank decomposition to train a rescoring BERT model and adapt it to new domains using only a fraction (0.08%) of the pretrained parameters. These inserted matrices are optimized through a discriminative training objective along with a correlation-based regularization loss. The proposed low-rank adaptation Rescore-BERT (LoRB) architecture is evaluated on LibriSpeech and internal datasets with decreased training times by factors between 5.4 and 3.6..

LoRAConfig

class peft.LoraConfig

< source >

```
( task_type: typing.Union[str, peft.utils.peft_types.TaskType, NoneType] = None, peft_type: typing.Union[str, peft.utils.peft_types.PeftType, NoneType] = None, auto_mapping: typing.Optional[dict] = None, base_model_name_or_path: typing.Optional[str] = None, revision: typing.Optional[str] = None, inference_mode: bool = False, r: int = 8, target_modules: Optional[Union[list[str], str]] = None, exclude_modules: Optional[Union[list[str], str]] = None, lora_alpha: int = 8, lora_dropout: float = 0.0, fan_in_fan_out: bool = False, bias: Literal['none', 'all', 'lora_only'] = 'none', use_rslora: bool = False, modules_to_save: Optional[list[str]] = None, init_lora_weights: bool | Literal['gaussian', 'eva', 'olora', 'pissa', 'pissa_niter_[number of iters]', 'corda', 'loftq'] = True, layers_to_transform: Optional[Union[list[int], int]] = None, layers_pattern: Optional[Union[list[str], str]] = None, rank_pattern: Optional[dict] = <factory>, alpha_pattern: Optional[dict] = <factory>, megatron_config: Optional[dict] = None, megatron_core: Optional[str] = 'megatron.core', trainable_token_indices: Optional[Union[list[int], dict[str, list[int]]]] = None, loftq_config: Union[LoftQConfig, dict] = <factory>, eva_config: Optional[EvaConfig] = None, corda_config: Optional[CordaConfig] = None, use_dora:
```